Basic Concepts Of Fixed Income: Basic Bond Concepts and the Term Structure of Interest Rates

Financial Python

Abstract

An introduction to bond pricing, and the term structure of interest rates.

1. Getting Started

1.a. Interacting With Jupyter Books

1.a.i. *Working with Jupyter Books:*

This and other chapters of *Basic Concepts of Fixed Income*, contain two types of documents:

1. **Markdown documents (*.md*):**
   - What you are currently reading.
   - Contain no executable code.
   - Can be downloaded as a Markdown file (*.md*) or a PDF file (*.pdf*).
2. **Notebook documents (*.ipynb*):**
   - HTML Representations of Jupyter notebooks.
   - Can be downloaded as a notebook (*.ipynb*) or a PDF file (*.pdf*).
   - Have interactive capabilities.
   - A notebook can be run inside the HTML file or in binder with Jupyter Lab.

Notebook Launch Options

There are three interactive options for notebook documents:

Option	Description	Interactive Features	Note
Binder	Launches in a new browser tab (typically within a minute) and runs remotely. Supports up to 100 simultaneous users.	Execute existing cells, add or modify cells, and download the notebook.	The notebook will not be saved or available if saved in this mode.

Published Apr 07, 2026

Option	Description	Interactive Features	Note
Colab	Offers the quickest and easiest access, not requiring Binder. A Google Drive is unnecessary for launching.	Execute cells or make a copy.	You must sign in to a Google account to execute cells or make a copy. You can download the notebook without signing into a Google account.
Live Code	Requires a Binder launch and is ideal for demonstrations.	Run code in existing cells, and alter code in existing cells.	You cannot add cells or save results in this mode.

The start button ⏻ in a notebook page allows you to execute the code live inside the HTML page.

The first cell in each notebook includes options to open in Colab or with Jupyter Lab in Binder.

Additional Interactive Feature

Copying code: Any code cell or code block can be copied to the clipboard by moving the cursor to its upper-right corner.

Summary

Jupyter books effectively combine Markdown and notebooks to highlight financial concepts with real-world applications.

2. Basic Concepts of Fixed Income: Chapter One

2.a. Chapter Highlights

2.a.i. *Basic Bond Concepts and the Term Structure of Interest Rates:*

This Jupyter Book is Chapter One of the volume Basic Concepts of Fixed Income.

Chapter One introduces the foundational principles of bond pricing. A primary focus of this chapter is calculating the term structure of interest rates, an essential gauge of financial market health, using data from US Treasury Bills (T-bills).

Highlights of the chapter are:

- ***Financial concepts***
  - *Basic bond pricing*
  - *Defining the term structure of interest rates*
  - *Calculating and graphing the term structure of interest rates*
  - *The term structure and the state of financial markets*
- ***Python concepts***
  - *NumPy arrays*
  - *Pandas DataFrames*
  - *Accessing data with a URL address*
  - *Custom modules*

Background:

This chapter's examples and discussions rely on the **Pandas** and **NumPy** libraries.

- **Pandas** is introduced in *A Quick Introduction to Pandas*.
- **NumPy** is introduced in *A Quick Introduction to NumPy*.
- The *NumPy and Pandas* notebook in this chapter provides examples of using these libraries in relation to the *Calculating and Graphing the Term Structure of Interest Rates* notebook.
- Additional relevant Python concepts can be found in the introductory volume, *Background Material: An Introduction to Python for Financial Python*, that relate to this and other chapters of *Basic Concepts of Fixed Income*.

The chapter includes five sections:

1. *Introduction to Bond Pricing* covers some basic concepts.
2. The Jupyter notebook *NumPy and Pandas* introduces and illustrates the modules.
3. The Jupyter notebook *Calculating and Graphing the Term Structure of Interest Rates* uses NumPy and Pandas to:
   - download data from DropBox that is used to calculate and graph the term structure on September 15, 2022.

- download data from the U.S. Treasury that is used to calculate and graph the term structure on September 30, 2025
- an exercise asking you to calculate and plot spot and forward rates with data downloaded from the US Treasury.

4. *Python, Pandas, NumPy, and Financial Insights* summarizes the financial concepts and results.
5. *Functions Imported by the Chapter* describes the functions imported from DropBox (*module_basic_concepts_fixed_income*).

2.b. Bond Pricing

2.b.i. *Introduction to Bond Pricing:*

This chapter will introduce some fundamental principles of fixed income. These principles include pricing coupon bonds as a portfolio of zero-coupon bonds, the term structure of interest rates, and the relation between spot and forward rates of interest.

The present value of a bond:

Chapter One of the volume *Time Value Of Money* asserted that the value of an asset with certain cash flows equals the present value of its cash flows, as shown in the following equation, where T is the total number of cash flow periods, and $PV(t)$ is the present-value factor for a certain cash flow in period t:

$$Value = \sum_{t=1}^{T} Cash\,Flow_t \times PV(t) \tag{1}$$

In finance, equating value with present value is ubiquitous and seems unworthy of much thought. It is, however, a powerful statement that value depends upon the ability to replicate an asset with other assets. Namely, if an asset's cash flow dates and amounts can be replicated by another asset or combinations of assets, the values must be equal. You might call this idea the *law of one price*—a concept that is key in all pricing models.

Coupon bonds and zero-coupon bonds:

Most bonds, but not all, pay fixed coupons in each period and principal at their maturity. The values of coupon bonds are calculated as:

$$Coupon\,Bond = \sum_{t=1}^{T} Coupon_t \times PV(t) + Principal_T \times PV(T) \tag{2}$$

Some bonds only repay principal at maturity, or T. Because there are no periodic coupon payments, these bonds are appropriately known as zero-coupon bonds. The values of zero-coupon bonds are calculated as:

$$Zero\,Coupon\,Bond = Principal_T \times PV(T) \tag{3}$$

In order to price any bond, it's necessary to know the present value of payments at the relevant dates. If the zero-coupon bond values are known for the relevant dates, the present value of each date can be determined. To do so, dvide the value of the zero-coupon bond by its principal, as shown below:

$$PV(t) = \frac{Zero\,Coupon\,Bond_t}{Principal_t} \tag{4}$$

Each coupon-bond cash flow is equivalent to an investment in a zero-coupon bond. As such, we can replicate a coupon bond with a sequence of zero-coupon bonds: The number of zero-coupon bonds with a maturity of t equals the coupon bond's cash flow at t divided by the zero-coupon bond's principal. For example, consider a $40 coupon that is paid every six months before maturity. This coupon is valued with a

sequence of 0.04 zero-coupon bonds, with a $\$1,000$ principal and maturities matching the coupon payment dates. The coupon bond's payment at maturity equals 1.040 zero-coupon bonds, with a $\$1,000$ principal and the same maturity as the coupon bond. Further, a coupon bond's value is the sum of all the zero-coupon bond values needed to replicate that bond's cash flows. If the coupon bond and the zero-coupon bonds have the same principal, and all the coupon payments of the coupon bond are equal, the value of the coupon bond is:

$$Value = \sum_{t=1}^{T} Zero\,Coupon\,Bond_t \times \frac{Coupon}{Principal} + Zero\,Coupon\,Bond_T \tag{5}$$

We define the sum as:

$$S(T) = \sum_{t=1}^{T} \frac{Zero\,Coupon\,Bond_t}{Principal} = \sum_{t=1}^{T} PV(t) \tag{6}$$

And the value of the fixed-coupon bond is composed of two components equaling:

$$Value = S(T) \times Coupon + PV(T) \times Principal(T) \tag{7}$$

Arbitrage and the value of the bond:

Thus far, bond values have been represented as the sum of the zero-coupon bonds' values. But we must consider the zero-coupon bonds' *market prices*—not their "theoretical" values. Will the value relation between zero-coupon and coupon bonds hold if we use the market prices of zero-coupon bonds? To answer that question, let's look at examples where the coupon bond is worth more or less than the sum of the zero-coupon bonds. First, consider a case where the coupon bond's price exceeds the sum of the zero-coupon bond prices:

$$Value > S(T) \times Coupon + PV(T) \times Principal(T) \tag{8}$$

If the price of the coupon bond exceeds the sum of the zero-coupon bond prices, the coupon bond can be *shorted*.[1] The zero-coupon bonds can then be purchased with the proceeds, leaving the investor with identical cash flows for a lower price. Consider if the inequality is reversed:

$$Value < S(T) \times Coupon + PV(T) \times Principal(T) \tag{9}$$

In this case, the *zero-coupon bonds* can be shorted, and the proceeds can be used to buy the coupon bond. This ability to recreate cash flows allows for arbitrage between zero-coupon and coupon bonds; as a result, the coupon bond prices equal the costs of the zero-coupon bonds used to recreate their cash flows. It is the law of one price in action. Whatever the bond prices might have been in theory does not matter. Those prices were ideas—the market value is a fact. This arbitrage relationship is important for deriving the *term of structure of interest rates* and *forward rates*, which we'll examine later in this chapter. These concepts are key to understanding bond pricing as well as some important fixed-income derivatives.

---

[1] The mechanics of shorting bonds will be discussed in future chapters.

🤖 **AI Study Assistant-Bond Pricing**

Select a prompt below to open Google AI and explore these concepts further ↗
(Sign in to save your chat history!)

✨ **Why is there an inverse relation between interest rates and bond prices? ↗**

✨ **Help with present value for coupon bonds ↗**

✨ **How is the present value formula related to arbitrage?↗**

Continuous and discrete compounding:

For mathematical and computational convenience, our discussion of fixed income and bond pricing will use continuously compounded interest rates. At times, it's appropriate to quote interest rates as discretely compounded rates. To that end, this short aside demonstrates the conversion between continuously and discretely compounded rates.

Suppose that R_c is the continuously compounded annual interest rate for T periods, and R_d is the discretely compounded annual rate for T periods when compounded d times per year. The conversion of continuously and discretely compounded rates requires that the same values are realized in T periods, shown below:

$$e^{R_c \times T} = \left(1 + \frac{R_d}{d}\right)^{d \times T} \tag{10}$$

e is Euler's number and is the limit of $\left(1 + \frac{1}{d}\right)^d$ as d approaches infinity or continuously compounds. The necessary condition is expressed as an annualized interest rate by taking the T root of both sides of the equation:

$$e^{R_c} = \left(1 + \frac{R_d}{d}\right)^d \tag{11}$$

Taking natural logs of the previous equation derives continuously compounded rates from discretely compounded rates:

$$R_c = d \times ln\left(1 + \frac{R_d}{d}\right) \tag{12}$$

Finally, discretely compounded rates can be derived from continuously compounded rates by dividing both sides by the compounding frequency, or

$$\frac{R_c}{d} = ln\left(1 + \frac{R_d}{d}\right) \tag{13}$$

and raising Euler's number, e, to both sides of the equation:

$$R_d = d \times \left(e^{\frac{R_c}{d}} - 1\right) \tag{14}$$

For instances in which rate conversion is necessary, the above methods are useful knowledge; for this chapter, however, we'll focus on continuously compounded rates.

📖 **Chapter Summary: The Fundamentals of Bond Pricing**

In a nutshell: This chapter explains that the value of any bond is simply the sum of its future cash flows, discounted to the present.

Key Takeaways:

- **The Law of One Price:** A coupon bond is essentially a "package" of zero-coupon bonds. If the price of the package differs from the sum of its parts, an **arbitrage** opportunity exists.
- **Replication:** We can replicate a coupon bond's cash flows by purchasing specific amounts of zero-coupon bonds. The market price of a bond must equal this replication cost.
- **Present Value (PV):** The value of any bond is calculated using:

$$Value = \sum_{t=1}^{T} Cash\,Flow_t \times PV(t) \tag{15}$$

- **Compounding Conversions:** While discrete compounding is common in practice, continuous compounding (using e) is often used for mathematical elegance. You can convert between them using: $R_c = d \times \ln\left(1 + \frac{R_d}{d}\right)$

Why this matters: Understanding that bonds are just bundles of zero-coupon payments allows us to price complex fixed-income derivatives and understand the term structure of interest rates.

2.c. Numpy and Pandas

2.c.i. *NumPy and Pandas:*

Open in Colab

🚀 Launch Live in Binder⬅️ Return to Main Book

[▾] How to use this page: Run, Copy, & Download

- ⏻ **Run code right here:** Click the **Power Button** icon at the top of the screen to activate **Live Code**.
- 📋 **Copy code:** Hover over any code block and click the **Clipboard icon** in the top-right corner.
- 📥 **Download this file:** Click the **Download icon** (downward arrow) at the top right of the screen to save this exact notebook to your computer.

🛠️ Notebook Setup: Why the "Try/Except" Imports?

The Goal: To ensure this notebook runs perfectly whether you are using **Google Colab**, a local **Jupyter instance**, or a remote server without you having to manually install software.

- **External Libraries:** NumPy and Pandas are the "heavy hitters" for data. They aren't always installed by default.
- **The `try/except` Logic:** This is a safety net.
    1. We **try** to import the library.
    2. If it fails (because it's not installed), the **except** block triggers a `!pip install` to download it automatically.
- **Aliasing (`as np`):** We rename `numpy` to `np` to save keystrokes. In professional finance code, `np` and `pd` are the universal shorthand.

Preparing the Notebook:

Importing libraries and functions:

If the attempted import within the `try` segment fails, the `except` portion will install the libraries using `pip`, a package installer for Python. The exclamation mark preceding `pip` indicates that `pip` is running through the machine's console rather than within the Notebook. Upon successful import, NumPy and Pandas are *aliased as*

(or assigned the names of) `np` and `pd`, respectively (for more information on these statements, see Control Statements).

```
try:
    import numpy as np
except:
    !pip install numpy
    import numpy as np
try:
    import pandas as pd
except:
    !pip install pandas
    import pandas as pd
```


```
try:
    import numpy as np
except:
    !pip install numpy
    import numpy as np
try:
    import pandas as pd
except:
    !pip install pandas
    import pandas as pd
```


🔢 **Why NumPy for Fixed Income?**

The Mental Model: Think of a NumPy **Array** as a high-powered version of a Python list. While standard Python lists are for general data, NumPy arrays are designed for **linear algebra and vector calculus**.

What to look for in this section:

- **Vectorization:** Notice how we apply a single formula (like `np.exp`) to an entire list of rates at once. In finance, this allows us to price 100 different bonds as easily as one.
- **Dimensionality:** We will move from a 1D list of rates to a 2D "Matrix" that pairs rates with their corresponding Present Value factors.
- **Efficiency:** NumPy performs these calculations in C-extension memory, making it thousands of times faster than standard Python loops for large datasets.

NumPy Arrays:

The NumPy library offers efficient numerical procedures that simplify complex calculations(for more information see, A Quick Introduction To NumPy).The fundamental object in this library is the NumPy array. An array is simply a data structure in which elements are arranged and accessed. It can be formed from a single value or from an iterable data type like a list, which holds a series of values (see A

First Look at Lists).NumPy must be imported before use; recall that in this Notebook, it was imported and aliased as `np` in the first code cell. The `array()` method is used to convert variables into NumPy arrays (**the allias np precedes the method array**) . The following code creates an array from the list `[3%, 5%, 7%]` (lists are stylized with square brackets) and assigns that array to `interest_rates`. This is then passed to `display()`, which simply displays the contents of `interest_rates`.

```
interest_rates=np.array([0.03,0.05,0.07])
display(interest_rates)
```

```
array([0.03, 0.05, 0.07])
```

NumPy Calculations:

Now that we've created an array and assigned it to `interest_rates`, we can perform calculations on that array. NumPy calculates array elements individually. For instance, the `exp()` method used below raises Euler's number, *e*, to the power of each element of `interest_rates` resulting in future value factors. This approach is faster and more convenient than iterating through the rates.

```
display(np.exp(interest_rates))
```

```
array([1.03045453, 1.0512711 , 1.07250818])
```

✍️ *Application: Create a NumPy Array:*

The Challenge: Create a new NumPy array containing the **present-value factors** calculated from the rates in `interest_rates`.

Recall: PV = e^{-r*t}. Use the vectorized power of NumPy to do this in a single line!

Two-Dimensional NumPy arrays:

The `pv_rates` array in the code below is a *two-dimensional* NumPy array (or matrix). It's created by combining two one-dimensional arrays—`interest_rates` and `pv_factors`—within the `array()` method. To access the rows of this two-dimensional array, we'll use the corresponding row *index* value. (The index value is simply an element's sequential position in the array. Remember that in programming, sequences begin at position 0, not 1). Finally, both the interest rates and present-value factors are displayed below using *f-strings*. These are like strings, but begin with an "f" and are used to include variables and expressions within a string. The variable must be enclosed in curly brackets (i.e., { }), and the indexed value must be enclosed in square brackets ([ ]).

```
#A calculation is make for each element of a numpy array
pv_factors=1/np.exp(interest_rates)
pv_rates=np.array([interest_rates,pv_factors])
display(pv_rates)
#Create an f string for each row.
#Each row must also be encapsulated in curly brackets
```

```python
display(f'Interest Rates {pv_rates[0]}')
display(f'Present Value Factors {pv_rates[1]}')
```

```
array([[0.03      , 0.05      , 0.07      ],
       [0.97044553, 0.95122942, 0.93239382]])
```

```
'Interest Rates [0.03 0.05 0.07]'
```

```
'Present Value Factors [0.97044553 0.95122942 0.93239382]'
```

✍️ *Application: Two-Dimensional Matrix Manipulation:*

The Challenge: Create a new two-dimensional array by performing a specialized calculation on your existing data.

1. Take the **interest-rate row** from your matrix.
2. Raise it to Euler's number (e^r).
3. Multiply the resulting values by the **present value factors**.

Goal: This verifies the relationship between discount factors and compounding rates within a single NumPy object.

🐼 **Why Pandas for Fixed Income?**

The Mental Model: If NumPy is the "engine" that does the math, **Pandas** is the "dashboard" that makes the data readable. It turns raw matrices into labeled tables called **DataFrames**.

Key Advantages:

- **Labeling:** Instead of remembering that "Row 1 is the PV Factor," we can give columns names like `Interest_Rate` and `PV_Factor`.
- **Data Alignment:** Pandas ensures that if you sort your rates, the corresponding PV factors stay perfectly aligned with them.
- **Readability:** DataFrames render as clean, professional HTML tables in your Jupyter Book, making them much easier for clients or researchers to audit.

Creating a Pandas DataFrame from NumPy Arrays:

A simple example:

The Pandas library is a powerful resource for data analysis. Recall that like NumPy, we imported Pandas in our earlier code cell. For this section, we'll be using Pandas to access a structure called a *DataFrame*. A Pandas DataFrame organizes data into rows and columns, much like an Excel spreadsheet or a NumPy array. It can be created from a two-dimensional array, such as our newly created `pv_rates`. The simplest DataFrames automatically assign row and column index values; a more functional approach allows for the explicit naming of both. In the simple example below, the `DataFrame()` method of Pandas uses three arrays as arguments, generating a row for each array.

```
# Dataframe created with DataFrame() method
simple_example=pd.DataFrame((interest_rates,
              np.exp(interest_rates),
              1/np.exp(interest_rates)))
display(simple_example)
```

	0	1	2
0	0.030000	0.050000	0.070000
1	1.030455	1.051271	1.072508
2	0.970446	0.951229	0.932394

🐼 *Application: Building a Pandas DataFrame:*

The Challenge: Convert your two-dimensional `pv_rates` array into a structured **Pandas DataFrame**.

- **Goal:** Transform the raw matrix into a labeled table that clearly identifies which values are interest rates and which are present-value factors.
- **Pro-Tip:** Remember that DataFrames often prefer data in columns. You may need to **Transpose** your array using `.T` during the creation process!

Making rows into columns and labeling the columns:

In the below code, rows are converted to columns by using the `transpose()` method on our earlier DataFrame, `simple_example`. This method changes the rows into columns (see A Quick Introduction To Pandas). The transposed result is then assigned to `rows_to_columns`, and the columns are then assigned the elements of a list (`['Rates', 'Future Value', 'Present Value']`). Transposing the DataFrame makes `'Rates'`, `'Future Value'`, and `'Present Value'` the variables and makes the rows the observations of those variables.

```
#transpose the simple_example dataframe and call it rows_to_columns
rows_to_columns=simple_example.transpose()
#(the transpose method can also be written as simple_example.T)
rows_to_columns.columns=['Rates','Future Value','Present Value']
display(rows_to_columns)
```

	Rates	Future Value	Present Value
0	0.03	1.030455	0.970446
1	0.05	1.051271	0.951229
2	0.07	1.072508	0.932394

The set_index() method:

Once the DataFrame has been transposed, the next step is to make the `Rates` column the index. Why? The reason is that the values of `Rates` determine the values of the columns `Future Value` and `Pesent Value`. This change isn't necessary, but it draws

our eye to a natural connection. Instead of an index that points only to a row number, the new index points to the rate that results in the future and present values.

The method `set_index()` takes a column name and removes the column from the DataFrame (an example). Pandas has a built-in *fail-safe-switch*. By default, this change (and others we'll encounter later) doesn't permanently alter the DataFrame. This might seem a bit strange, but this characteristic prevents mistakes that are difficult to fix. To make the change permanent, you can do the following:

1. Assign the transposed value of the DataFrame to another DataFrame *

```
name_index_set_index=rows_to_columns.set_index('Rates')
display(name_index_set)
```

1. Change the default value of the `inplace` argument of `set_index` to True *

```
rows_to_columns.set_index('Rates',inplace=True)
display(rows_to_columns)
```

```
name_index_set_index=rows_to_columns.set_index('Rates')
display(name_index_set_index)
display(rows_to_columns)
rows_to_columns.set_index('Rates',inplace=True)
display(rows_to_columns)
```

	Future Value	Present Value
Rates		
0.03	1.030455	0.970446
0.05	1.051271	0.951229
0.07	1.072508	0.932394

	Rates	Future Value	Present Value
0	0.03	1.030455	0.970446
1	0.05	1.051271	0.951229
2	0.07	1.072508	0.932394

	Future Value	Present Value
Rates		
0.03	1.030455	0.970446
0.05	1.051271	0.951229
0.07	1.072508	0.932394

A final check:

Open in Colab

🚀 Launch Live in Binder⬅️ Return to Main Book

2.d. Calculating and Graphing The Term Structure

2.d.i. *Calculating and Graphing the Term Structure of Interest Rates:*

Open in Colab

🚀 Launch Live in Binder⬅️ Return to Main Book

[▾] How to use this page: Run, Copy, & Download

- ⏻ **Run code right here:** Click the **Power Button** icon at the top of the screen to activate **Live Code**.
- 📋 **Copy code:** Hover over any code block and click the **Clipboard icon** in the top-right corner.
- 📥 **Download this file:** Click the **Download icon** (downward arrow) at the top right of the screen to save this exact notebook to your computer.

🛠️ Notebook Setup: Why the "Try/Except" Imports?

The Goal: To ensure this notebook runs perfectly whether you are using **Google Colab**, a local **Jupyter instance**, or a remote server without you having to manually install software.

Key Concepts in this Section:

- **Standard Libraries:** Modules like `os`, `sys`, and `datetime` come "in the box" with Python. We use them for system tasks and date math.
- **External Libraries:** NumPy and Pandas are the "heavy hitters" for data. They aren't always installed by default.
- **The `try/except` Logic:** This is a safety net.
    1. We **try** to import the library.
    2. If it fails (because it's not installed), the **except** block triggers a `!pip install` to download it automatically.
- **Aliasing (`as np`):** We rename `numpy` to `np` to save keystrokes. In professional finance code, `np` and `pd` are the universal shorthand.

Preparing the Notebook:

Importing libraries, modules, and functions:

It's good practice to import all required libraries and modules in the first code cell of a Notebook.

The standard Python installation includes five module imports: `os`, `sys`, `requests`, `types`, and `datetime`. The imports `os` and `sys` facilitate interaction with the operating system. The `requests` module enables sending and retrieving data from external URLs; in this notebook, the module is specifically used to access files from Dropbox. The `types` module lets us create the module from the Python code accessed from Dropbox. Finally, `datetime` allows for the creation and manipulation of date objects.

The code imports these modules, and specifically accesses `types` and `datetime` to import `ModuleType` and `date`, respectively. `ModuleType` will allow us to *instantiate* new modules, and we'll use `date` for our date calculations later.

```
import os
import sys
import requests
from types import ModuleType
from datetime import date
```

If the attempted import within the `try` segment fails, the `except` portion will install the libraries using `pip`, a package installer for Python. The exclamation mark preceding `pip` indicates that `pip` is running through the machine's console rather than within the Notebook. Upon successful import, NumPy and Pandas are *aliased as* (or assigned the names of) `np` and `pd`, respectively (for more information on these statements, see Control Statements).

```
try:
    import numpy as np
except:
    !pip install numpy
    import numpy as np
try:
    import pandas as pd
except:
    !pip install pandas
    import pandas as pd
```

```
import os
import sys
import requests
from types import ModuleType
try:
    import numpy as np
except:
    !pip install numpy
    import numpy as np
try:
    import pandas as pd
except:
```

```
!pip install pandas
import pandas as pd
```

> ☁️ **Cloud-Loading: How In-Memory Modules Work**
>
> **The Logic:** Usually, Python looks for modules as `.py` files on your hard drive. Here, we are "tricking" Python into treating a string of text from a URL as a live library.
>
> **The Workflow:**
>
> 1. **Fetch:** `requests.get(url)` grabs the raw text of your Python script from Dropbox.
> 2. **Instantiate:** `ModuleType(module_name)` creates an empty "container" in your computer's RAM.
> 3. **Execute:** `exec(code, module.__dict__)` runs that text inside the container, turning text into live functions.
> 4. **Register:** By adding it to `sys.modules`, we tell Python: *"If I try to import this later, don't look on the disk—look right here in the memory."*
>
> **Why do this?** It makes your notebooks **100% portable**. A user can open this in a brand-new environment, and as long as they have an internet connection, all your custom financial functions will "just work."

Adding a custom module and importing functions:

Now that we've imported our main modules and libraries, we'll access a custom module for our more specific functions. In the code below, the custom module `module_basic_concepts_fixed_income` contains functions utilized by this notebook and others in the volume Basic Concepts of Fixed Income.

We access this module from Dropbox using `requests.get()`. This approach allows the notebook to remain "portable"—it fetches the necessary tools directly from the cloud without requiring you to manage local files.

The module is instantiated with `ModuleType` (imported earlier). Once created, the module becomes accessible in the Notebook's memory, though it is not saved to your hard drive. The `exec()` function then executes the Python code returned by the URL and assigns it to `sys.module`, making the functions ready for use.

Finally, we import the function one_y_axis, which we will use to plot our calculation results.

```
try:
    response = requests.get(url)
    module = ModuleType(module_name)
    exec(response.text, module.__dict__)
    sys.modules[module_name] = module
    # Now we can import from our in-memory module
```

```
    from module_basic_concepts_fixed_income import one_y_axis
except requests.exceptions.RequestException as e:
    print(f"❌ Error: Could not fetch module from URL. {e}")
except Exception as e:
    print(f"❌ Error: Failed to execute or import the module. {e}")
```

Our now-complete code is shown in the cell below.

```
# Define the URL of the Python module to be downloaded from Dropbox.
# The 'dl=1' parameter in the URL forces a direct download of the file
content.
url= 'https://www.dropbox.com/scl/fi/4y5hjxlfphh1ngvbgo77q/\
module_basic_concepts_fixed_income.py?
rlkey=6oxi7mgka42veaat79hcv8boz&st=stuou11h&dl=1'
module_name='basic_concepts_fixed_income'
# Send an HTTP GET request to the URL and store the server's response.
try:
    response = requests.get(url)
    module = ModuleType(module_name)
    exec(response.text, module.__dict__)
    sys.modules[module_name] = module
    # Now we can import from our in-memory module
    from basic_concepts_fixed_income import one_y_axis
except requests.exceptions.RequestException as e:
    print(f"❌ Error: Could not fetch module from URL. {e}")
except Exception as e:
    print(f"❌ Error: Failed to execute or import the module. {e}")
```

Term Structure: Putting NumPy and Pandas to Work:

The *term structure of interest rates* (term structure) describes how interest rates vary over time. It's a critical financial metric reflected in the pricing or valuation of almost all assets.

NumPy and Pandas are powerful tools that greatly simplify the manipulation and calculation of extensive and complex financial data. In this section, we'll explore their application in calculating the term structure on various dates, a process that requires accessing large datasets and performing numerous calculations.

Calculating term structure:

The earlier discussion of bond pricing described the price of a zero-coupon bond as its maturity value multiplied by the present-value factor. A NumPy array and a Pandas DataFrame included interest rates, future values, and present-value factors. These same concepts are used to calculate term structure.

Spot rates of interest:

Put simply, term structure is the relation between the annualized interest rates of zero-coupon bonds, or the *spot rate* of interest, and the bonds' maturities. The price of a zero-coupon bond with a maturity of t and a principal of $1 is

$$P(t) = e^{-r_t \times t} \tag{16}$$

The annualized continuously-compounded interest rate of the t-period zero-coupon bond is:

$$r_t = \frac{-ln(P(t))}{t} \tag{17}$$

with r_t representing the spot rate of interest for t periods. As such, if a zero-coupon bond pays $1 in six months and has a price of $0.9753, the spot rate of interest is:

$$5\% = \frac{-ln(0.9753)}{0.5} \tag{18}$$

The spot rate reflects a return that begins immediately for a specific period of time. To consider future periods, we turn to *forward rates*.

Forward rates of interest:

The forward rate of interest represents the additional return gained by extending an investment's duration—or the extra earnings from investing for two months instead of one, for example. When two bonds offer an identical single payment at maturity, the forward rate between the shorter and longer maturities is determined by the ratio of their respective prices.

For zero-coupon bonds with a principal of $1, thecalculationofoneplustheforwardratebetweentandt+\Delta t$ is:

$$\frac{\frac{1}{P(t+\Delta t)}}{\frac{1}{P(t)}} = e^{f(t,t+\Delta t)\times\Delta t} \tag{19}$$

simplified to:

$$\frac{P(t)}{P(t+\Delta t)} = e^{f(t,t+\Delta t)\times\Delta t} \tag{20}$$

The continuously-compounded annualized forward rate between t and $t + \Delta t$ is:

$$f(t, t+\Delta t) = \frac{\ln(P(t)) - \ln(P(t+\Delta t))}{\Delta t} \tag{21}$$

The forward rates can be locked in by purchasing a zero-coupon bond that matures in $t + \Delta t$ periods and for t periods, borrowing $P(t + \Delta t)$ today at the rate $P(t)^{-1}$. At t, the borrowing is repaid, and at $t + \Delta t$, the zero-coupon bond matures and pays $1. The borrowing, repayment, and locked-in rate-of-return calculations are shown below:

$$\text{Amount Borrowed} = P(t+\Delta t) \tag{22}$$

$$\text{Repayment} = P(t+\Delta t) \times \frac{1}{P(t)} \tag{23}$$

$$\text{Locked -In Rate of Return} = \frac{1}{\frac{P(t+\Delta t)}{P(t)}} - 1 = \frac{P(t)}{P(t+\Delta t)} - 1 \tag{24}$$

The continuously-compounded annualized locked-in rate of return equals:

$$\frac{\ln(P(t)) - \ln(P(t+\Delta t))}{\Delta t} \tag{25}$$

This value is the same as the continuously-compounded forward rate of interest between t and $t + \Delta t$.

Although it might seem obvious that the locked-in rate of return and the forward rate are equal, this result is significant. Imagine an investor who intends to invest at the future date t for Δt periods. The investor can:

1. Wait until t to invest at the unknown spot rate $r(t, t + \Delta t)$, or
2. Lock in a rate of return today that equals the current forward rate $f(t, t + \Delta t)$.

In a state of equilibrium, the prices of zero-coupon bonds ensure that investors are indifferent to either securing a spot rate immediately or deferring investment until a future, unknown spot rate. Similar to the odds in a sports wager, the forward rate represents a "fair-bet" value for these future unknown spot rates. Forward rates forecast future spot rates, but that's not the same as equaling the expected values.

Section Summary

This upcoming section bridges foundational financial theory with practical Python programming by demonstrating how to analyze the term structure of interest rates using Treasury bills (T-bills). It serves as a practical guide for extracting and manipulating real-world financial data.

Key topics covered include:

Core Financial Concepts: An introduction to T-bills—short-term securities that make a single principal payment at maturity—and why they are priced based on single spot rates. It also highlights the term structure of interest rates as a vital economic indicator that reflects past, present, and future market conditions.

Importing Financial Data: A practical walkthrough of using the Pandas library to fetch data directly from the web. You will learn how to use the read_excel() method to seamlessly convert an online Excel workbook into a usable Pandas DataFrame.

Data Manipulation and Retrieval: A detailed look at how to navigate your newly created DataFrames. The section explains how to pinpoint and extract specific rows of data using Pandas indexing tools, specifically the .loc (label-based) and .iloc (integer-based) attributes, as well as the .get_indexer() method for handling missing values gracefully.

> By the end of this section, you will understand both the economic significance of T-bill term structures and the essential Python data-wrangling techniques required to access and analyze them effectively.

Calculating the Term Structure of Interest Rates with Treasury Bills:

Treasury bills, or T-bills, mature in a year or less and make a single principal payment at maturity. In this section, we'll apply the Python concepts we've learned so far to T-bill data, using the bills to demonstrate the term structure between one month and one year. Because the bills make a single payment, their prices are determined by the single spot rate of interest at maturity.

Python and finance in action:

This example will outline two Python concepts integral to financial calculations. First, it will demonstrate how to leverage the Pandas library to access Excel workbooks from a URL and convert them into DataFrames—a highly effective skill given Excel's widespread use in the finance world. The Pandas method we'll use is `read_excel`. Second, this example will illustrate the valuable data manipulation techniques available through Pandas, specifically focusing on how the attributes `loc` and `iloc` and the method `get_indexer` are used to retrieve the necessary data for calculations (Pandas and Excel, loc and iloc atributes).

Measuring the term structure is a key financial skill. The term structure of interest rates is a critical indicator of financial market conditions. Its explanation, along with the concepts necessary for its calculation, represents the most significant aspect of this example. The relationships between past, present,and future market conditions are all expressed by measuring the term structure.

Getting T-bill data:

T-bill data is available from the US Department of the Treasury (Treasury) at the department's Daily Treasury Bill Rate page. The data used in this example is for September 2022 and was uploaded to the authors' Dropbox as an Excel workbook.

Accessing the data:

The Pandas magic used to read an Excel workbook from a URL is `read_excel()`. The arguments of the method are:

1. The website address, or `url`. This argument is a required item
2. The columns used as the index, or `index_col`. This argument is optional and is assigned to the `Date` column.

The below code assigns the Dropbox URL to `url` and sets `index_col` to `Date`, then passes both through our `read_excel()` method to give us `Tbill_data_excel`.

```
url='https://www.dropbox.com/scl/fi/wdb24k0dddkpq7qry5sl8/\
daily-treasury-rates-9-22.xlsx?rlkey=qxoo8vpa4iziu5kvi27u6c41l&dl=1'
Tbill_data_excel=pd.read_excel(url, index_col='Date')
```

Creating a DataFrame from an Excel file:

Let's now turn this snippet into useable code. In the following code cell, an Excel workbook is accessed from Dropbox, which provides the URL. The Pandas method `read_excel()` is used to read and assign the data to `Tbill_data_excel`. The `Date` column is assigned as the index. Notice here that `try` and `except` are used to alert us if the file is not available.

```
#The full file path.
url='https://www.dropbox.com/scl/fi/wdb24k0dddkpq7qry5sl8/\
daily-treasury-rates-9-22.xlsx?rlkey=qxoo8vpa4iziu5kvi27u6c41l&dl=1'
print(f"Attempting to load data from: {url}")

#Load the data from Excel, using the Date column as the index.
try:
    Tbill_data_excel = pd.read_excel(url, index_col='Date')

    # Display the loaded DataFrame to verify it worked.
    display(Tbill_data_excel.head())

except FileNotFoundError:
    print("\nERROR: File not found.")
    print("Please check that the 'URL' address")
```

```
Attempting to load data from: https://www.dropbox.com/scl/fi/wdb24k0
dddkpq7qry5sl8/daily-treasury-rates-9-22.xlsx?rlkey=qxoo8vpa4iziu5kvi27u
6c41l&dl=1
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call
last)
Cell In[1], line 13
      9
     10     # Display the loaded DataFrame to verify it worked.
     11     display(Tbill_data_excel.head())
     12
---> 13 except FileNotFoundError:
     14     print("\nERROR: File not found.")
     15     print("Please check that the 'URL' address")

NameError: name 'pd' is not defined
```

Accessing rows of a Pandas DataFrame:

We've now created a DataFrame, but we still need to access its contents. Three common ways of accessing the rows of a DataFrame are the attributes `loc` (as in "location") and `iloc` (as in "integer location") and the method `get_indexer()`. All three ways access rows by the value of the index.

The `loc` attribute of a DataFrame accesses a row by using its index value. In the below code, the specific value of the index `'09/15/2022'` is accessed. If the value is not in the index, an error is returned via `except`. There are only 30 days in September, so `'09/31/2022'` fails and the message `'Value Not in Index'` is displayed.

```
try:
  display(Tbill_data_excel.loc['09/15/2022'])
except:
  display('Value Not in Index')
try:
  display(Tbill_data_excel.loc['09/31/2022'])
except:
  display('Value Not in Index')
```


```
try:
  display(Tbill_data_excel.loc['09/15/2022'])
except:
  display('Value Not in Index')
try:
  display(Tbill_data_excel.loc['09/31/2022'])
except:
  display('Value Not in Index')
```


	2022-09-15
4 WEEKS BANK DISCOUNT	2.68
4 WEEKS COUPON EQUIVALENT	2.72
8 WEEKS BANK DISCOUNT	2.92
8 WEEKS COUPON EQUIVALENT	2.97
13 WEEKS BANK DISCOUNT	3.12
13 WEEKS COUPON EQUIVALENT	3.19
26 WEEKS BANK DISCOUNT	3.69
26 WEEKS COUPON EQUIVALENT	3.81
52 WEEKS BANK DISCOUNT	3.83
52 WEEKS COUPON EQUIVALENT	4.00

dtype: float64

```
'Value Not in Index'
```

The `iloc` attribute accesses a row by its indexed integer value. In this case, the `Date` column "09/15/2022" has an index value of 11. If the integer value is not present, an error is returned, as in the earlier code. The DataFrame has 21 rows, and so the index value of 45 fails, as shown below.

```
try:
  display(Tbill_data_excel.iloc[11])
except:
  display('Value Not in Index')
try:
  display(Tbill_data_excel.loc[45'])
except:
  display('Value Not in Index')
```

```
try:
  display(Tbill_data_excel.iloc[11])
except:
  display('Value Not in Index')
try:
  display(Tbill_data_excel.iloc[45])
except:
  display('Value Not in Index')
```

	2022-09-15
4 WEEKS BANK DISCOUNT	2.68
4 WEEKS COUPON EQUIVALENT	2.72
8 WEEKS BANK DISCOUNT	2.92
8 WEEKS COUPON EQUIVALENT	2.97
13 WEEKS BANK DISCOUNT	3.12
13 WEEKS COUPON EQUIVALENT	3.19
26 WEEKS BANK DISCOUNT	3.69
26 WEEKS COUPON EQUIVALENT	3.81
52 WEEKS BANK DISCOUNT	3.83
52 WEEKS COUPON EQUIVALENT	4.00

dtype: float64

```
'Value Not in Index'
```

Rounding out our Pandas tools, the method `get_indexer()` returns the indexed integer values for a specific value of the index as an array. In the below example, an array with a single value is returned. For the Date "09/15/2022" the value 11 is returned. The `get_indexer()` method returns −1 for missing values, which avoids interrupting the code.

```
displayT(bill_data_excel.index.get_indexer(['09/15/2022'])
displayT(bill_data_excel.index.get_indexer(['09/31/2022'])
```

```
#return row number for date '09/15/2022'
display(Tbill_data_excel.index.get_indexer(['09/15/2022']))
#September has only 30 days, value of -1 returned
display(Tbill_data_excel.index.get_indexer(['09/31/2022']))
```

```
array([11])
```

```
array([-1])
```

Calculating the Term Structure: A Return:

Having gathered the bond data, we can return to calculating the term structure. Recall that the price of a T-bill (a zero-coupon bond) is its principal discounted back at the annualized continuous spot rate of interest. The forward rate of interest is the difference of the natural logs of the prices of a shorter maturity and a longer maturity zero-coupon bond. To calculate the term structure, we need to first calculate the prices of the T-bills (the zero-coupon prices).

T-bills are quoted as bank discounts or coupon-equivalent yields.

$$CBill\,Price = 100 - bank\,discount \times \frac{Maturity\,In\,Days}{360} \tag{26}$$

The bank discount of the four-week bill on September 15th is 2.68%, which implies a bill price of:

$$99.792 = 100 - 2.68 \times \frac{28}{360} \tag{27}$$

The coupon equivalent is the annualized rate of return that is calculated with the convention of *365 days* per year. The calculation is the actual rate of return (the gain on the bill divided by the bill price) multiplied by 365 divided by the maturity of the bill in days. A four-week bill matures in 28 days, and its coupon equivalent is:

$$Coupon\,Equivalent = \frac{100 - Bill\,Price}{Bill\,Price} \times \frac{365}{28} \tag{28}$$

And a coupon equivalent of:

$$2.72\% = \frac{100 - 99.792}{99.79} \times \frac{365}{28} \tag{29}$$

Calculating bond prices with bank discounts:

In the `Tbill_data_excel` DataFrame, columns are either `*BANK DISCOUNT*` or `*COUPON EQUIVALENT*`. We'll construct a NumPy array `bank_discount` from the `*BANK DISCOUNT*` columns. In the following code, the list `desired_columns` is created with list comprehension from the columns attribute of `Tbill_data_excel`. If the string `'BANK'` is in the column name, that column name is added to the list `desired_columns`.[5]

```
desired_columns=[column for column in Tbill_data_excel.columns if 'BANK'
in column]
```

The column is only added when the `if` statement is true. The `df_bankers_discount` DataFrame is created by including the `desired_column` lists as the columns of the `Tbill_data_excel` DataFrame, as shown below.

```
df_bankers_discount=Tbill_data_excel[desired_columns]
display(df_bankers_discount[:5])
```

Having the bank discount in NumPy arrays will allow for an easy conversion to bill prices. We can convert `df_bankers_discount` into a two-dimensional NumPy array `bankers_discount` with the Pandas method `to_numpy()`. The first five rows of the array are shown below (see Pandas to NUmPy)

```
bankers_discount=df_bankers_discount.to_numpy()
display(bankers_discount[:5])
```

```
# Create a list called 'desired_columns' using a list comprehension.
desired_columns = [column for column in Tbill_data_excel.columns if
'BANK' in column]

# Create a DataFrame named 'df_bankers_discount' and 'bankers_discont'.
df_bankers_discount = Tbill_data_excel[desired_columns]
bankers_discount=df_bankers_discount.to_numpy()
# Display the first 5 rows of the newly created 'df_bankers_discount'
DataFrame.
display(df_bankers_discount[:5])
display(bankers_discount[:5])
```

	4 WEEKS BANK DISCOUNT	8 WEEKS BANK DISCOUNT	13 WEEKS BANK DISCOUNT	26 WEEKS BANK DISCOUNT	52 WEEKS BANK DISCOUNT
Date					
2022-09-30	2.70	3.07	3.22	3.82	3.87
2022-09-29	2.68	3.08	3.26	3.78	3.80
2022-09-28	2.50	2.96	3.29	3.78	3.81
2022-09-27	2.60	2.99	3.26	3.83	3.97
2022-09-26	2.64	2.99	3.30	3.87	3.98

```
array([[2.7 , 3.07, 3.22, 3.82, 3.87],
       [2.68, 3.08, 3.26, 3.78, 3.8 ],
       [2.5 , 2.96, 3.29, 3.78, 3.81],
       [2.6 , 2.99, 3.26, 3.83, 3.97],
       [2.64, 2.99, 3.3 , 3.87, 3.98]])
```

Going from bankers' discounts to zero prices:

As noted earlier, the price of a bill with a $100 par value is:

$$Bill\ Price = 100 - bank\ discount \times \frac{Maturity\ In\ Days}{360} \tag{30}$$

The price of one dollar, or the the zero price, is:

$$\frac{100 - bank\,discount \times \frac{Maturity\,In\,Days}{360}}{100} \tag{31}$$

The maturity of the bills are 4, 8, 13, 26, and 52 weeks. In the following code, the NumPy array `maturity_days` is created by multiplying the number of weeks by seven. The `zero_price_one` array is calculated by multiplying `bankers_discount` by `maturity_days`, annualized with 360 days per year. We then divide by 100 to convert the value to a one dollar zero price.

```
maturity_days=np.array([4*7,8*7,13*7,26*7,52*7]) #Maturity in days
calculated
#The price formula divides maturity days by 360 to annualize
#Then divides by to convert to a bill paying one dollar.
zero_prices_one=(100-bankers_discount*maturity_days/360)/100
#NumPy performs element by element calculations: a calculation for each
#value of maturity_days with the corresponding value of bankers_discount
zero_prices_one[:5]   #Display first 5 rows (each row is a trading day
for Sept 2022)
```

```
array([[0.9979    , 0.99522444, 0.99186056, 0.98068778, 0.96087   ],
       [0.99791556, 0.99520889, 0.99175944, 0.98089   , 0.96157778],
       [0.99805556, 0.99539556, 0.99168361, 0.98089   , 0.96147667],
       [0.99797778, 0.99534889, 0.99175944, 0.98063722, 0.95985889],
       [0.99794667, 0.99534889, 0.99165833, 0.980435  , 0.95975778]])
```

Spot rates:

Recall that the spot rates are the annualized rates calculated from the T-bill data. Spot rates are equal to:

$$Spot\ \ Rate(T) = \frac{ln(zero\ \ price)}{\frac{T}{365}} \tag{32}$$

where T is the number of days. In the following code, the array `spot_rates` is calculated from the `zero_prices_one` array. The first five rows of the array are then displayed.

```
spot_rates=-np.log(zero_prices_one)/(maturity_days/365) #Maturity_days
divided by 365
spot_rates[:5] #Display first 5 rows (last 5 trading days of Sept 2022)
```

```
array([[0.02740378, 0.03120095, 0.03278081, 0.03910943, 0.04002581],
       [0.02720058, 0.03130283, 0.03318972, 0.03869593, 0.03928746],
       [0.0253719 , 0.03008042, 0.03349642, 0.03869593, 0.03939291],
       [0.0263878 , 0.030386  , 0.03318972, 0.03921282, 0.04108155],
       [0.02679418, 0.030386  , 0.03359866, 0.03962642, 0.04118718]])
```

Forward rates:

The forward rates are calculated with differences in the natural logs of zero prices and assigned to the matrix `forward_rates` in the below code. The NumPy method

`diff()` is used to calculate the difference between the zero prices for adjacent maturities. The arguments of `diff()` are the array name and the number of lags between differences. The default value for lags between differences is 1.

```
forward_rates=-np.diff(np.log(zero_price_one))*365/
np.diff(maturity_days)
```

The above line of code warrants a bit of explanation. The `diff()` method subtracts the lower indexed value from the higher—in this example, the maturity of four weeks from eight weeks and eight weeks from thirteen and so on. The calculation of forward rates is the reverse (or the negative) value of the `diff()` method. This negative value is assigned to the `forward_rates` array. The `diff()` function returns four values (there is no forward rate for the first month). To align the spot and forward rates on the graph, a dummy value, `nan`, is inserted in the forward-rate array. The NumPy method `insert()` is used to add a column with `nan` as the first column of the `forward_rates` array. As you might have guessed, `nan` stands for "not a number," and it's ignored in calculations. The arguments of `insert()` are the array, position, value, and axis. The rows are `axis 0`, and the columns are `axis 1`. This means that a position of 0 and an axis of 1 inserts the values as the first column.

```
forward_rates=-np.insert(forward_rates,0,np.nan,axis=1)
```

The first five values of the `forward_rates` array are shown below.

```
forward_rates=-np.diff(np.log(zero_prices_one))*365/
np.diff(maturity_days) #Sign switched
forward_rates=np.insert(forward_rates,0,np.nan,axis=1) #NumPy nan
inserted for first month
forward_rates[:5] #Display first five dates
```

```
array([[       nan, 0.03499811, 0.03530859, 0.04543804, 0.0409422 ],
       [       nan, 0.03540507, 0.03620874, 0.04420214, 0.039879  ],
       [       nan, 0.03478894, 0.03896203, 0.04389543, 0.04008989],
       [       nan, 0.03438419, 0.03767567, 0.04523591, 0.04295028],
       [       nan, 0.03397781, 0.03873893, 0.04565418, 0.04274794]])
```

Saving Results:

We suggest saving files (graphs, spreadsheets, and so forth) generated by notebooks. To that end, a parameter of the `one_y_axis` function is the dictionary `save_config`. The keys of the dictionary are assumed to be:

- `volume`
- `chapter`
- `file_name`

If values are assigned to at least one of the keys, the graph will be saved. If you change the name of the keys, the value will be ignored. In this example the dictionary

is created with the constructor `dict`. Dictionaries can also be createded with {keys:values} (see backgroundon dictionaries.

```
save_config=dict(file_name='Term Structure September 15 2022.pdf',
                 volume='Basic Concepts Fixed Income',
                 chapter= 'One')
```


```
save_config=dict(file_name='Term Structure September 15 2022.pdf',
                 volume='Basic Concepts Fixed Income',
                 chapter= 'One')
```


Graphing the spot and forward rates for September 15, 2022:

In the following code, the *x*-axis is set at the five maturities of the bills in months. Forward and spot rates are calculated for the month of September 2022, and the data for September 15, 2022, is plotted.

```
xaxis=['One','Two','Three','Six','Twelve']
string_date='09/15/2022'
```

Since they lack index columns, the correct rows of the NumPy arrays `spot_rates` and `forward_rates` cannot be determined from the value of `string_date`. Because the rows of the arrays correspond to the rows of the DataFrame, the correct row is determined with our now familiar Pandas method `get_indexer()`. The method returns an array with index values corresponding to the argument. The spot- and forward-rate rows for September 15, 2022, are then multiplied by 100 and assigned to the list `data`.

```
loc=Tbill_data.index.get_indexer([string_date])[0]
data=[spot_rates[loc]*100,forward_rates[loc]*100]
```

The plot title is assigned whatever value is assigned to `save_config['File Name']` (minus the PDF extension). If no title, it's assigned "Add A Title."

```
try:
  title = os.path.splitext(save_config['file_name'])
except:
  title='Add A Title'
```

Series names, labels, and markers are then assigned.

```
series = ['Spot Rate', 'Forward Rate']
ylabel = 'Continuous Compounded Annualized % Rates'
xlabel = 'Maturity In Months'
x = len(data)
markers = ['<','>']*round(x/2+x%2+0.001)
```

The upper limit of the *y*-axis is assigned to `upper` and calculated from the maximum value of the spot and forward rates. The NumPy method `nanmax()` ignores `nan` values. The maximum of `nanmax()` of spot rates (`data[0]`) and forward rates (`data[1]`) plus 0.5% are assigned to `upper` . The lower limit is set to 0, and the size

and colors of the lines are set as seven inches wide and five inches tall (7,5) and red and blue, respectively.

```
lower = 0
upper = max(np.nanmax(data[0]), np.nanmax(data[1])) + 0.5
ylim = [lower, upper]
size = (7, 5)
colors=['r','b']
```

Finally, the graph is generated and saved with `one_y_axis`.

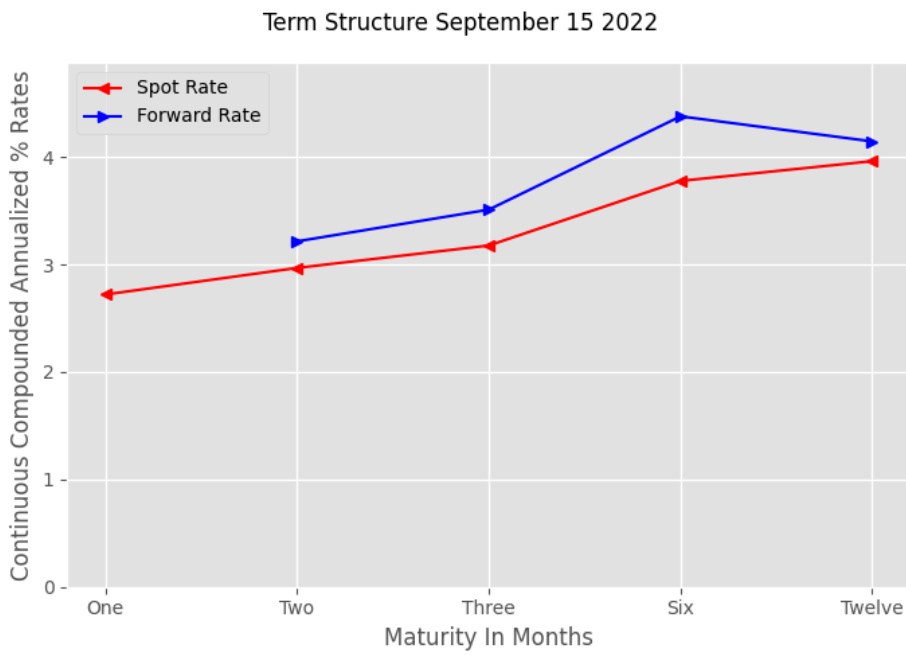
```
one_y_axis(xaxis, data, title, series, xlabel, ylabel, markers, size,
ylim,
           save_config=save_config,colors=colors)
```

```
save_config=dict(file_name='Term Structure September 15 2022.pdf',
                 volume='Basic Concepts Fixed Income',
                 chapter= 'One')
# Define the labels for the x-axis, representing different maturities.
xaxis = ['One', 'Two', 'Three', 'Six', 'Twelve']

# Define the specific date for which the term structure will be plotted.
string_date = '09/15/2022'

# Get the integer location (row index) of the specified 'string_date' in
the DataFrame's index..
loc = Tbill_data_excel.index.get_indexer([string_date])[0]

# Prepare the data to be plotted on the y-axis (as %).
data = [spot_rates[loc]*100, forward_rates[loc]*100]

# Attempt to extract the plot title from the file name.
try:
    title = os.path.splitext(save_config['file_name'])[0]
#'file_name key not found set a default title.
except:
    title = 'September 15, 2022'

# Define the names for the data series that will appear in the plot's
legend.
series = ['Spot Rate', 'Forward Rate']

# Define the label for the y-axis.
ylabel = 'Continuous Compounded Annualized % Rates'

# Define the label for the x-axis.
xlabel = 'Maturity In Months'

#'data' contains two series (spot and forward rates).
x = len(data)

# Define markers for the plot.
markers = ['<','>']*round(x/2+x%2+0.001)
```

```
# Define the lower bound for the y-axis.
lower = 0

# Define the upper bound for the y-axis.
# considering NaN values (not a number) and adds a small buffer (0.5)
for padding.
upper = max(np.nanmax(data[0]), np.nanmax(data[1])) + 0.5

# Define the y-axis limits (minimum and maximum values).
ylim = [lower, upper]

# Define the size of the plot (width, height) in inches.
size = (7, 5)
# Define colors as red and blue
colors=['r','b']
# This function generates and possibly saves a plot.
one_y_axis(xaxis, data, title, series, xlabel, ylabel, markers, size,
ylim,
           save_config=save_config,colors=colors)
```

```
❓ Do you want to save the file? (y/n) (press enter for n): n
```

```
<IPython.core.display.Markdown object>
```

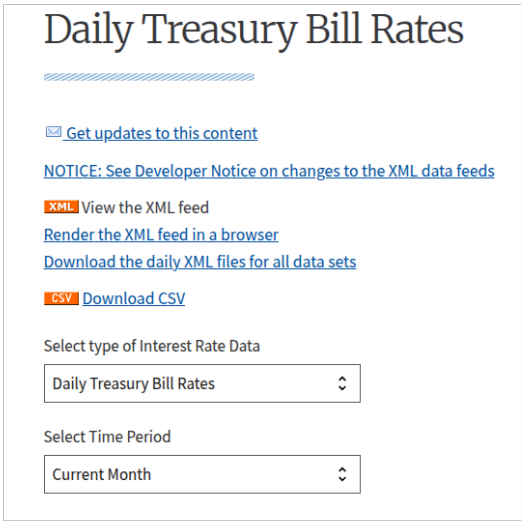


Accessing T-Bill Data Directly from the Treasury:

Figure 1 shows a screenshot of the Treasury page and its data tool, found here. The time period selected determines the URL for the "Download CSV" link. In our example, the year is set to 2025 and the month is September. As we will see below, the URL includes the string of a four-digit year and a two-digit month (`'202509'`).

The link to the Daily Treasury Bill Rates on the Treasury Site:

Figure 1. Accessing T-bill rates



Source: "Daily Treasury Bill Rates," US Department of the Treasury, 2025 (accessed Sept. 24, 2025)

Changing the URL and using the Pandas method `read_csv`:

The data is accessed directly from the Treasury as a CSV file, and the code below displays the data row for September 30, 2025. With the exception of the URL value and the `read_csv()` versus `read_excel()` methods, the code is identical to what we used to create `Tbill_data_excel`. In this example, the URL points to a Treasury site instead of Dropbox. The URL is copied by right-clicking the "Download CSV" link on the Treasury website. The URL for a particular year and month is shown below, and `data_date` is assigned the string `'202509'`. You can assign it any year and month after 2022.[9]

```
data_date='202509'
url='https://home.treasury.gov/resource-center/data-chart-center/\
interest-rates/daily-treasury-rates.csv/all/'+data_date+'?\
type=daily_treasury_bill_rates&field_tdr_date_value_month=+'data_date'+&page&_format=cs
```

The CSV file is in descending order of the dates. The first row is for September 30 and is displayed with `iloc[0]`.

[9]As of November 3, 2025, dates prior to 2022 will be moved to a historical page, according to the Treasury. See US Treasury Data Chart Center.

```
#Read the csv file from the U.S. Treasury website
#Construct the full file path.
data_date='202509'
url='https://home.treasury.gov/resource-center/data-chart-center/\
interest-rates/daily-treasury-rates.csv/all/'+data_date+'?\
type=daily_treasury_bill_rates&field_tdr_date_value_month='+data_date+'&page&_format=cs
print(f"Attempting to load data from {url}")

#Load the data from csv, using the Date column as the index.
try:
    Tbill_data_site = pd.read_csv(url,index_col='Date')
    # display September 30, 2025 data
    display(Tbill_data_site.iloc[0])

except FileNotFoundError:
    display("\nERROR: File not found.")
    display("Please check that the 'URL' address")
```

```
Attempting to load data from https://home.treasury.gov/resource-center/
data-chart-center/interest-rates/daily-treasury-rates.csv/all/202509?
type=daily_treasury_bill_rates&field_tdr_date_value_month=202509&page&_
format=csv
```

	09/30/2025
4 WEEKS BANK DISCOUNT	4.05
4 WEEKS COUPON EQUIVALENT	4.12
6 WEEKS BANK DISCOUNT	4.02
6 WEEKS COUPON EQUIVALENT	4.10
8 WEEKS BANK DISCOUNT	4.00
8 WEEKS COUPON EQUIVALENT	4.08
13 WEEKS BANK DISCOUNT	3.86
13 WEEKS COUPON EQUIVALENT	3.95
17 WEEKS BANK DISCOUNT	3.83
17 WEEKS COUPON EQUIVALENT	3.93
26 WEEKS BANK DISCOUNT	3.72
26 WEEKS COUPON EQUIVALENT	3.84
52 WEEKS BANK DISCOUNT	3.53
52 WEEKS COUPON EQUIVALENT	3.68

dtype: float64

For this date, the U.S. Treasury returns T-Bill rates for seven dates. In September of 2022, rates are returned for five dates.

Application: Downloading Treasury Data:

✍️ *Application: Download Treasury Data:*

The Challenge: Get data for September 12, 2024 from the U.S. Treasury site.

Recall: The url includes a string for the date.

Using the CSV file to calculate term structure:

The following sections proceed much the same as our earlier process for calculating the term structure and thus need little explanation. We'll convert banks' discounts to zero prices, calculate spot and forward rates, and prepare the Notebook for graphing the term structure.

```
# Create a list called 'desired_columns' using a list comprehension.
desired_columns = [column for column in Tbill_data_site.columns if
'BANK' in column]

# Create a DataFrame named 'df_bankers_discount'.
df_bankers_discount = Tbill_data_site[desired_columns]

# Display the first 5 rows of the newly created 'df_bankers_discount'
DataFrame.
df_bankers_discount[:5]
```

	4 WEEKS BANK DISCOUNT	6 WEEKS BANK DISCOUNT	8 WEEKS BANK DISCOUNT	13 WEEKS BANK DISCOUNT	17 WEEKS BANK DISCOUNT	26 WEEKS BANK DISCOUNT	52 WEEKS BANK DISCOUNT
Date							
09/30/2025	4.05	4.02	4.00	3.86	3.83	3.72	3.53
09/29/2025	4.06	4.03	4.00	3.86	3.83	3.73	3.52
09/26/2025	4.07	4.03	4.00	3.87	3.83	3.74	3.53
09/25/2025	4.08	4.03	4.01	3.89	3.84	3.75	3.54
09/24/2025	4.02	4.02	3.98	3.87	3.81	3.72	3.49

```
# The .to_numpy() method strips away the pandas Index and column
headers,
# leaving only the raw data values in a highly efficient array format.
bankers_discount=df_bankers_discount.to_numpy()

# --- Verification Step: Display the First 5 Rows of the New Array ---
# After a data type conversion, it is good practice to inspect the
result.
bankers_discount[:5]
```

```
array([[4.05, 4.02, 4.  , 3.86, 3.83, 3.72, 3.53],
       [4.06, 4.03, 4.  , 3.86, 3.83, 3.73, 3.52],
```

```
       [4.07, 4.03, 4.  , 3.87, 3.83, 3.74, 3.53],
       [4.08, 4.03, 4.01, 3.89, 3.84, 3.75, 3.54],
       [4.02, 4.02, 3.98, 3.87, 3.81, 3.72, 3.49]])
```

Going from bankers' discounts to zero prices:

```
maturity_days=np.array([4*7,6*7,8*7,13*7,17*7,26*7,52*7]) #Maturity in
days calculated
#The price formula divides maturity days by 360 to annualize
#Then divides by 100 to convert to a bill paying one dollar.
zero_prices_one=(100-bankers_discount*maturity_days/360)/100
#NumPy performs element by element calculations: a calculation for each
#value of maturity_days with the corresponding value of bankers_discount
zero_prices_one[:5]   #Display first 5 rows (each row is a trading day
for Sept 2025)
```

```
array([[0.99685   , 0.99531   , 0.99377778, 0.99024278, 0.98733972,
        0.98119333, 0.96430778],
       [0.99684222, 0.99529833, 0.99377778, 0.99024278, 0.98733972,
        0.98114278, 0.96440889],
       [0.99683444, 0.99529833, 0.99377778, 0.9902175 , 0.98733972,
        0.98109222, 0.96430778],
       [0.99682667, 0.99529833, 0.99376222, 0.99016694, 0.98730667,
        0.98104167, 0.96420667],
       [0.99687333, 0.99531   , 0.99380889, 0.9902175 , 0.98740583,
        0.98119333, 0.96471222]])
```

Spot rates:

```
spot_rates=-np.log(zero_prices_one)/(maturity_days/365) #Maturity_days
divided by 365
spot_rates[:5] #Display first 5 rows (last 5 trading days of Sept 2025)
```

```
array([[0.04112731, 0.04085421, 0.04068225, 0.03932829, 0.03907985,
        0.03807584, 0.03644461],
       [0.04122902, 0.04095608, 0.04068225, 0.03932829, 0.03907985,
        0.03817917, 0.03633948],
       [0.04133073, 0.04095608, 0.04068225, 0.03943068, 0.03907985,
        0.03828251, 0.03644461],
       [0.04143244, 0.04095608, 0.04078428, 0.03963547, 0.03918254,
        0.03838586, 0.03654976],
       [0.04082219, 0.04085421, 0.04047821, 0.03943068, 0.03887448,
        0.03807584, 0.03602413]])
```

Forward rates:

```
forward_rates=-np.diff(np.log(zero_prices_one))*365/
np.diff(maturity_days) #Sign switched
forward_rates=np.insert(forward_rates,0,np.nan,axis=1) #NumPy nan
inserted for first month
forward_rates[:5]
```

```
array([[       nan, 0.04030802, 0.04016638, 0.03716195, 0.03827242,
        0.03617937, 0.03481338],
       [       nan, 0.0404102 , 0.03986078, 0.03716195, 0.03827242,
```

```
 0.0364779 , 0.03449978],
[       nan, 0.04020678, 0.03986078, 0.03742816, 0.03793965,
 0.03677644, 0.03460671],
[       nan, 0.04000335, 0.04026888, 0.03779737, 0.03771053,
 0.03688102, 0.03471366],
[       nan, 0.04091826, 0.03935021, 0.03775464, 0.03706682,
 0.0365673 , 0.03397243]])
```

Graphing the Term Structure (Again):

```
save_config=dict(file_name='Term Structure September 30 2025.pdf',
                 volume='Basic Concepts Fixed Income',
                 chapter= 'One')
```

Graphing the spot and forward rates for September 30, 2025:

In the following code, the *x*-axis is set at the bills' seven maturities in months. Forward and spot rates are calculated for the month of September 2025, and the data for September 30, 2025, is plotted.

```
xaxis=['One','One & Half','Two','Three','Four','Six','Twelve']
string_date='09/15/2022'
```

The first row of `spot_rates` and `forward_rates` correspond to September 30. The spot- and forward-rate rows for September 30, 2025, are then multiplied by 100 and assigned to the list `data`.

```
data=[spot_rates[0]*100,forward_rates[0]*100]
```

The plot title is assigned whatever value is assigned to `save_config['File Name']` (minus the PDF extension).

```
try:
  title = os.path.splitext(save_config['file_name'])
except:
  title='September 30, 2025'
```

Series names, labels, and markers are then assigned.

```
series = ['Spot Rate', 'Forward Rate']
ylabel = 'Continuous Compounded Annualized % Rates'
xlabel = 'Maturity In Months'
x = len(data)
markers = ['<','>']*round(x/2+x%2+0.001)
```

The upper limit of the *y*-axis is assigned to `upper` and calculated from the maximum value of the spot and forward rates. As before, the NumPy method `nanmax()` ignores `nan` values. The maximum of `nanmax()` of spot rates (`data[0]`) and forward rates (`data[1]`) plus 0.5% are assigned to `upper` . The lower limit is set to 0, and the size and colors of the lines are once again set as 10 inches wide and 6 inches tall (10,6) and red and blue, respectively.

```
lower = 0
upper = max(np.nanmax(data[0]), np.nanmax(data[1])) + 0.5
ylim = [lower, upper]
size = (7, 5)
colors=['r','b']
```

Finally, the graph is generated and saved with `one_y_axis()`

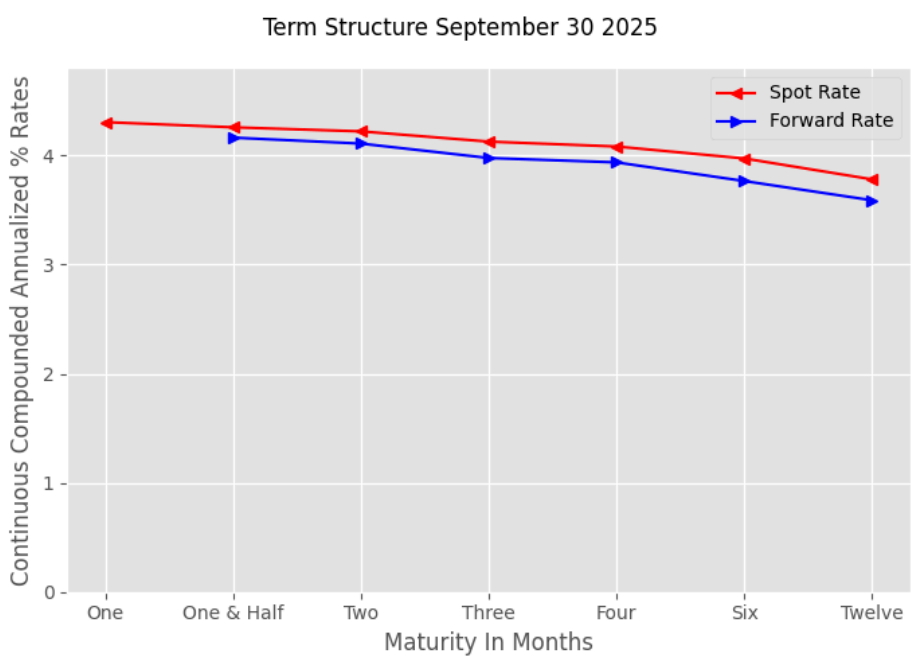
```
one_y_axis(xaxis, data, title, series, xlabel, ylabel, markers, size,
ylim,
           save_config=save_config,colors=colors)
```

```
# Define the labels for the x-axis, representing different maturities.
xaxis = ['One','One & Half', 'Two', 'Three','Four', 'Six', 'Twelve']

# Define the specific date for which the term structure will be plotted.
string_date = '09/30/2025'

# Get the integer location (row index) of the specified 'string_date' in
the DataFrame's index..
loc = Tbill_data_excel.index.get_indexer([string_date])[0]

# Prepare the data to be plotted on the y-axis (as %).
data = [spot_rates[loc]*100, forward_rates[loc]*100]

# Attempt to extract the plot title from the file name.
try:
    title = os.path.splitext(save_config['file_name'])[0]
#'file_name key not found set a default title.
except:
    title = 'September 30, 2025'

# Define the names for the data series that will appear in the plot's
legend.
series = ['Spot Rate', 'Forward Rate']

# Define the label for the y-axis.
ylabel = 'Continuous Compounded Annualized % Rates'

# Define the label for the x-axis.
xlabel = 'Maturity In Months'

#'data' contains two series (spot and forward rates).
x = len(data)

# Define markers for the plot.
markers = ['<','>']*round(x/2+x%2+0.001)

# Define the lower bound for the y-axis.
lower = 0

# Define the upper bound for the y-axis.
# considering NaN values (not a number) and adds a small buffer (0.5)
```

```
for padding.
upper = max(np.nanmax(data[0]), np.nanmax(data[1])) + 0.5

# Define the y-axis limits (minimum and maximum values).
ylim = [lower, upper]

# Define the size of the plot (width, height) in inches.
size = (7, 5)
# Define colors as red and blue
colors=['r','b']
# This function generates and possibly saves a plot.
one_y_axis(xaxis, data, title, series, xlabel, ylabel, markers, size,
ylim,
           save_config=save_config,colors=colors)
```

```
❓ Do you want to save the file? (y/n) (press enter for n): n
```

```
<IPython.core.display.Markdown object>
```



Open in Colab

🚀 Launch Live in Binder ⬅️ Return to Main Book

2.e. Chapter Summary

2.e.i. *Python, Pandas, NumPy, and Financial Insights:*

In this chapter, we used Pandas and NumPy to explore different ways of calculating the term structure of interest rates for September 15, 2022, and September 30, 2025. These term structures reveal an economy in transition. On September 15, 2022, rates were increasing with maturity and could be expected to continue rising. By September 30, 2025, however, the pattern had shifted. Although the overall level of rates was higher on September 30, 2025, the term structure showed a slight downward slope. These differences indicate two different economic states: one in which inflation was expected to increase (September 2022) and one in which it wasn't (September 2025). Calibrating the term structure of interest rates this way has highlighted Python's rich ecosystem of libraries and methods, and more importantly, shown us the language's utility for finance.

Summarizing key points:

The present value function describes all bonds as a portfolio of zero-coupon bonds.
Data was accessed directly from the Treasury as a CSV file and assigned to a Pandas Dataframe.
The term structure of interest rates describes the behavior of spot and forward interest rates
The term structure of interest rates was calculated with T-bills for September 2022.
The term structure was graphed for September 15, 2022, and September 30, 2025.

2.f. Imported Functions

2.f.i. *Functions Imported by the Chapter.:*

Chapter One's notebooks import two helper functions:

1. save_results: This function helps track files generated by notebooks that are saved.
2. one_y_axis: This function utilizes the matplotlib library to graph multiple series on a single y-axis.

Both functions are part of a module downloaded from DropBox. These and other functions within the module are intended to shift focus from coding to financial concepts.

3. Hub & Other Chapters Of Basic Concepts Of Fixed Income

3.a. Hub For Basic Concepts Of Fixed Income

3.b. Chapters Two: Accrued Interest

3.c. Chapters Three: Bond Pay Data

3.d. Chapters Four: Bootstrapping The Term Structure With Coupon Bonds

3.e. Chapter Five: Par Yields

3.f. Chapter Six: Yield To Maturity

3.g. Chapter Seven: Measuring Interest Rate Risk With Duration

3.h. Chapter Eight: Convexity And Interest Rate Risk

3.i. Chapter Nine: The Nelson and Siegel Model Of the Term Structure